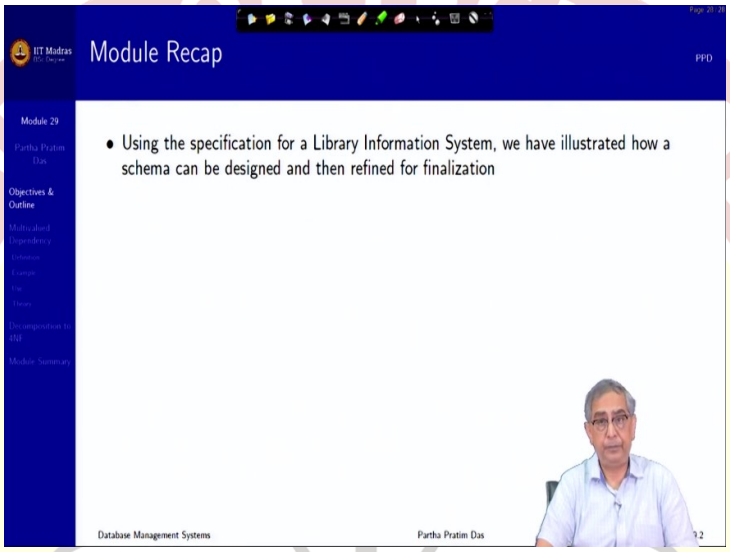
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pritam Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/9: MVD and 4NF

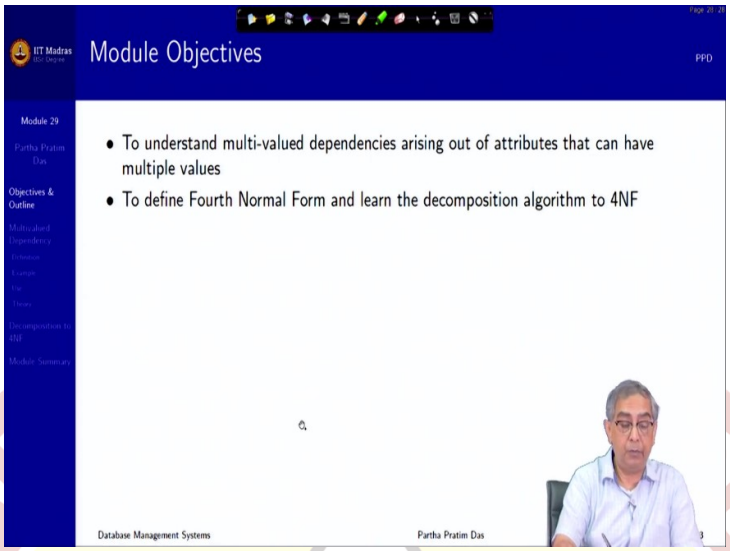
Welcome to Module 29 of Database Management Systems course in the IIT Madras online BSc program.

(Refer Slide Time: 00:28)



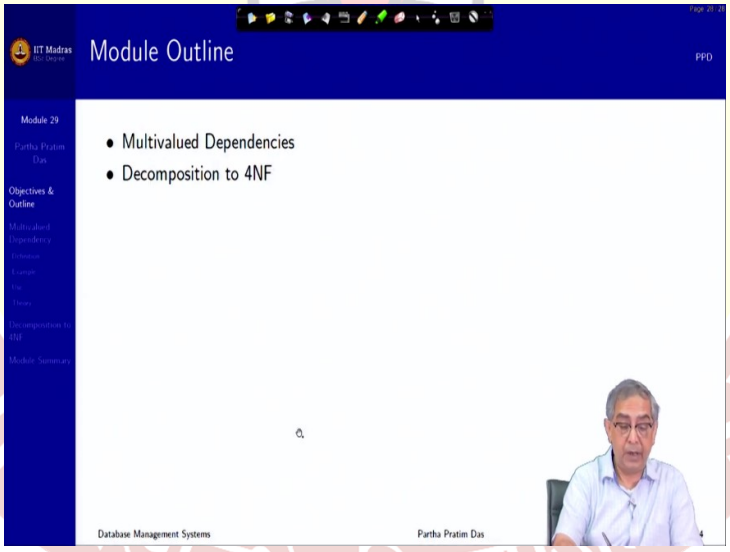
We have been discussing about Relational Database Design. We have done that through the functional dependency theory and normal forms and normalization and we culminated by discussing a specific Library Information System Design from specification to the relational schema, going through the application of each, I mean, different things that we have learned through the process. So, this brings us to kind of the learning for relational database design using functional dependency model.

(Refer Slide Time: 01:13)



Now, we are going to generalize this somewhat to take care of more forms of redundancy that may exist in the real world and we are going to talk about fourth normal form.

(Refer Slide Time: 01:29)

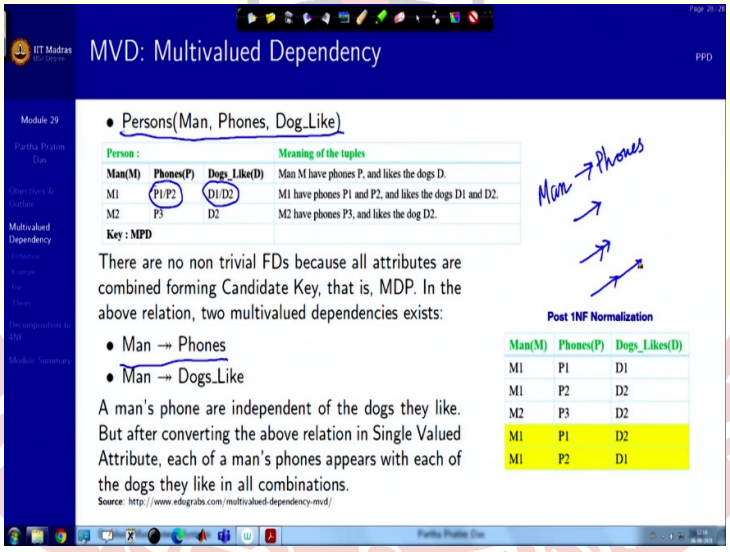


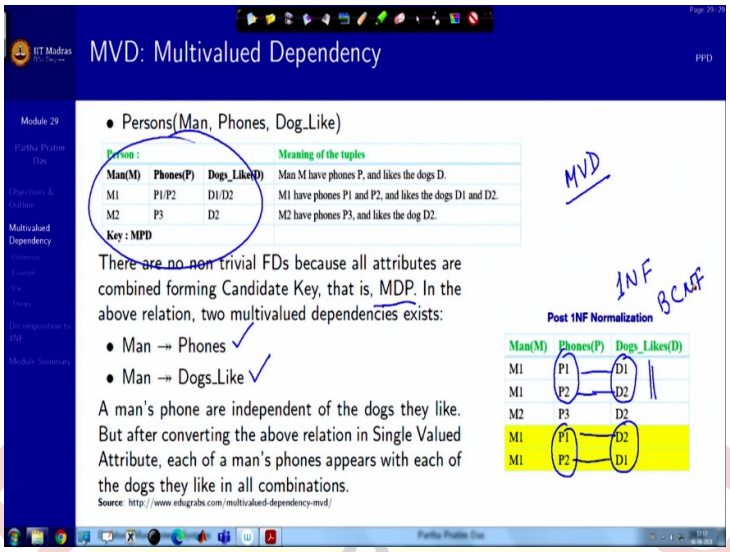
And before that the multivalued dependencies.

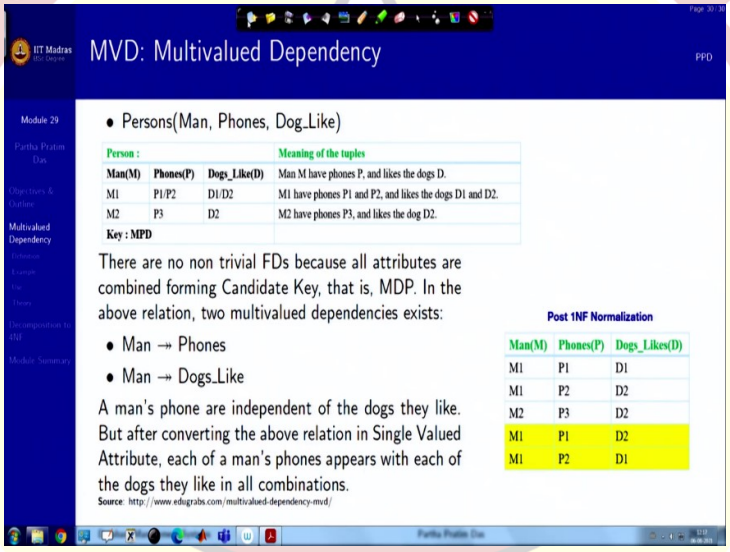
(Refer Slide Time: 01:30)

So, like we had functional dependencies where a set of attributes values over a set of attributes uniquely determines the values over another set of attributes, alpha determines beta.

(Refer Slide Time: 01:47)







We have a more generalized scenario if we have multivalued attributes. So, far what we have dealt with we have enforced that we start with 1NF, atomic single value. So, 1NF if we start with one 1NF every attribute can have a single value, but what if it can have multiple values we saw this situation earlier too, but now, we are going to go in depth. So, we are talking about a relationship, a relational schema person who has name, phone and likeness for dogs.

Naturally the person has multiple phones a person can have multiple phones and likeness for multiple dogs. So, these are the multiple values that are possible for that attribute. Now, if they were not possible then we would have said that man functionally determines phones. But now, we cannot say that anymore.

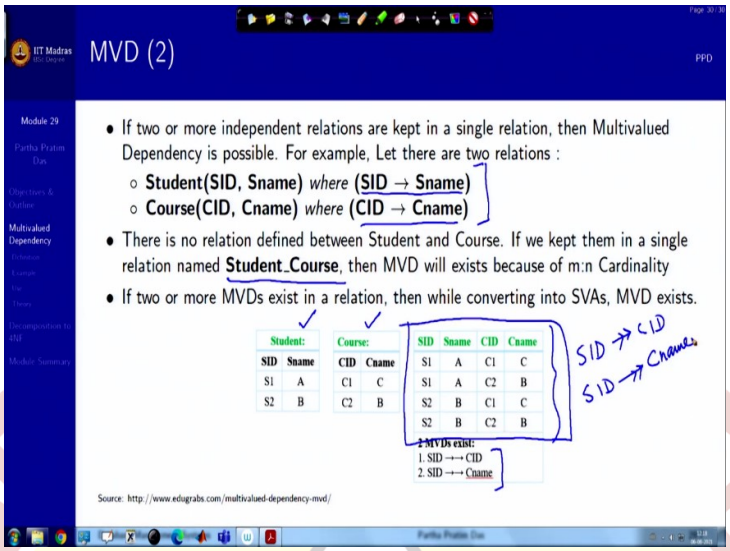
So, what is we say we now model this saying that man multi determines mind you earlier it was functionally determines now it is multi determines phones and the change in notation is functional dependency was this and this dependency is written with two head arrows at times people also write it like this. So, I have we have here two multi dependencies here multivalued dependencies here.

Multivalued dependencies in like functional dependency we say FD, multivalued dependencies we often say it is MVD. Now, if you look into this person, relational schema from the perspective of FDs there is no trivial FD and what would be the key for this? It will be all the attributes. Because nothing functionally determines anything else. So, the candidate keys man doglike phone MDP.

So, that is, that being the only key and there not being any non-trivial functional dependency, this relationship is already in BCNF. But if you look into this that there are multiple values. So, this, this relationship, relational schema is not even in 1NF, kind of contradiction. So, to make it into 1NF, what you will have to do, you will have to flatten out the multiple values as you start doing here P1 P2, P1 P2, D1 D2, D2 D1.

So, you have all possible combinations P1 D1, P1 D2, P2 D1, P2 D2, because once it gets single valued and you have to keep these values it gets into 1NF it actually still does not have any non-trivial functional dependency. So, it is in BCNF but, you can see what a tremendous amount of redundancy it has now. So, that is the new problem which cannot be handled by functional dependencies and has to be handled by the multivalued dependencies that we are going to talk about today.

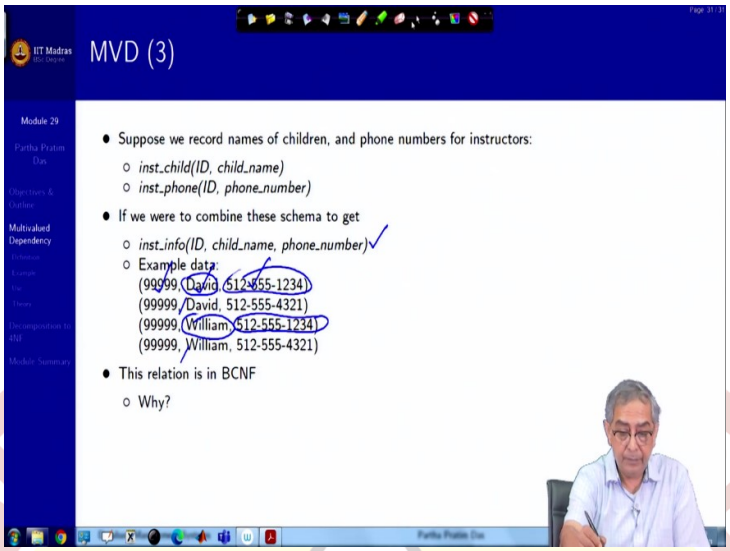
(Refer Slide Time: 06:08)



Now, if we have two independent relations, whenever we have two independent relations, say, we are talking about two independent relations here. One is student which is SID and Sname. Obvious functional dependency and other is course CID and Cname. If we have two independent relations, that is, I mean as such, there is nothing common between them I am nothing common in terms of attributes or any relationship.

But if we keep them in a single table let us say a student courses a student table is a course table, but if we keep them in a single table multivalued dependency will happen. Quite obviously, because, because since there is no relationship basically, when you keep them in the single table, what do you do? You do a join and in this joint, you just have a Cartesian product, there is no common attribute to filter. So, you get all of these and you get all this redundancy. And in this you have two multivalued dependencies SID multi determines CID and SID multi determines Cname. So, that is the.

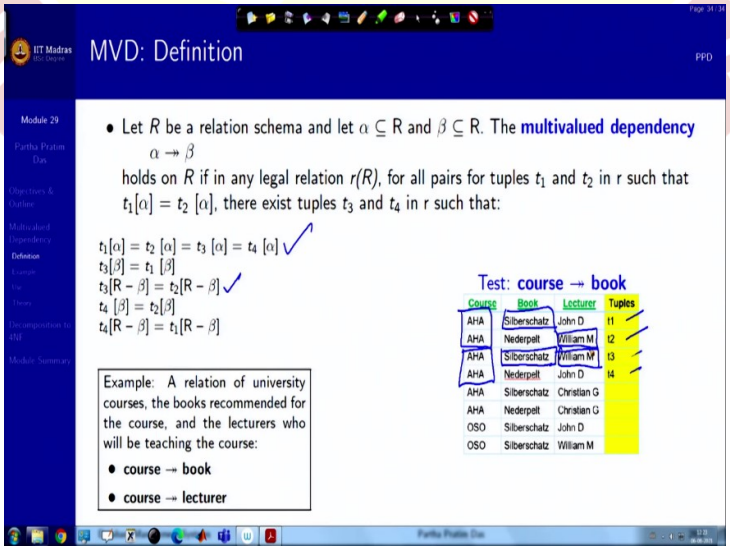
(Refer Slide Time: 07:50)

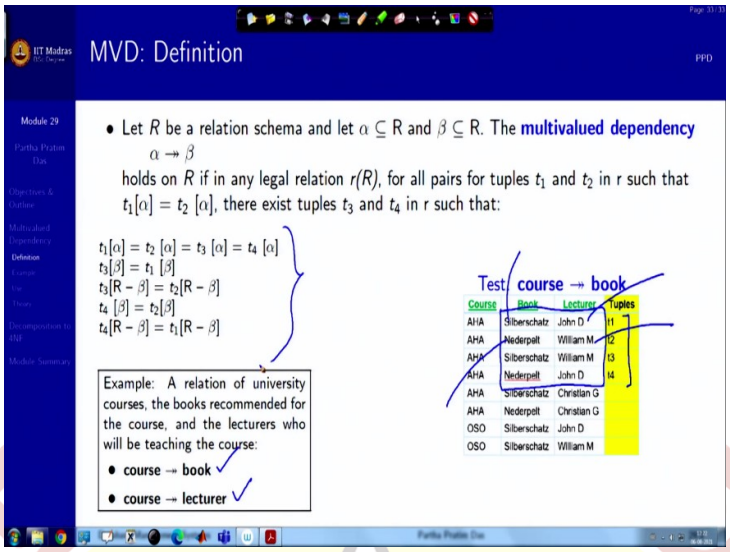


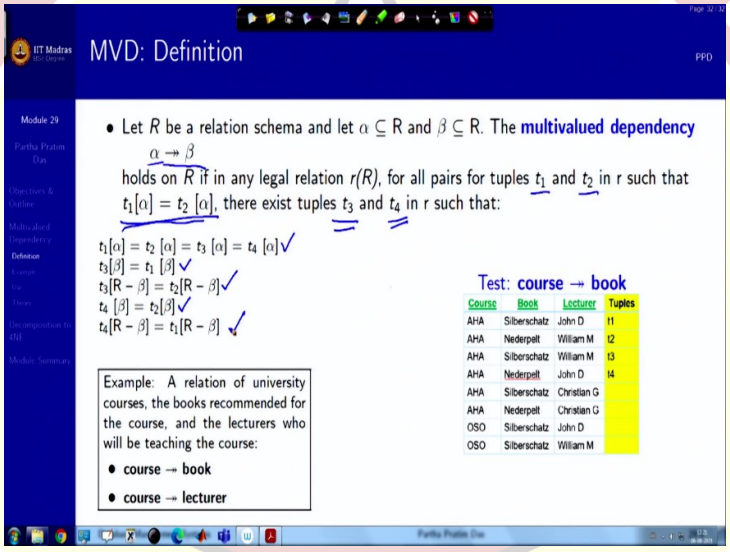
I mean this is just another perspective of the issue that example, I was trying to show. We talked about this example earlier also, this is instructor has multiple children, instructor's multiple phone numbers and keep them in different tables, but if you combine the schemas, if you have the instructors ID, you can go back to that module and see the details also.

If we combine them into instructors ID, the child's name and the phone number we will get one relationship where there is no violation of BCNF it is in BCNF but, we have MVDs, we have data duplication. So, you get to know that this person's child is David and William, but you record it twice this phone number you record twice and more and more you have reexplored. Different examples and perspective we just.

(Refer Slide Time: 09:12)







So, having understood the situation let us now formalize it like we did for the functional dependency. So, what you say is if R is a relational schema and alpha beta are subsets of R, then a multivalued dependency as we write holds on R if any legal relation so that is that is always a common statement. The for all pair of tuples t1 and t2 that match on alpha. In functional dependency, what do you say?

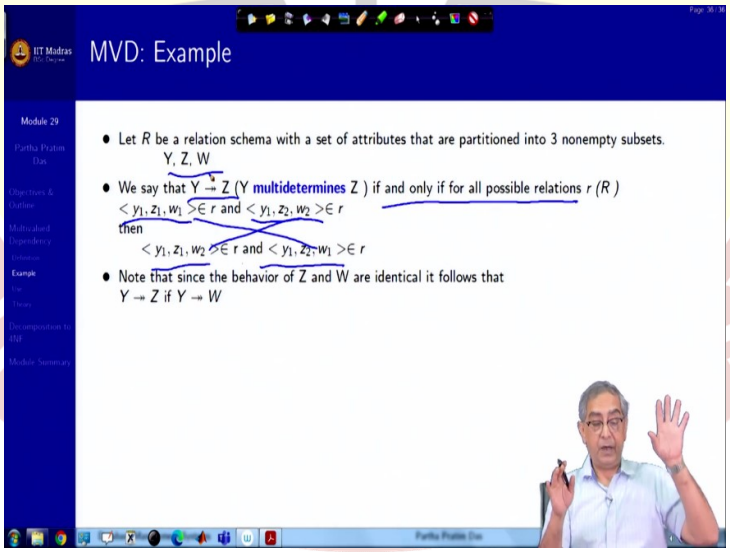
If they match on alpha, they will match on beta but it is but now it is not that it is a lot more explosive. If they match on alpha then there will exist two more tuple t3 and t4 such that all of them match on alpha, t3 and t1 match on beta, t3[R-beta] that is the remaining attributes match with t2s [R-beta], t4 beta matches with t2 beta and so on. All of these, looks complicated if you write down in mathematical form, but it is as such not that the example will help.

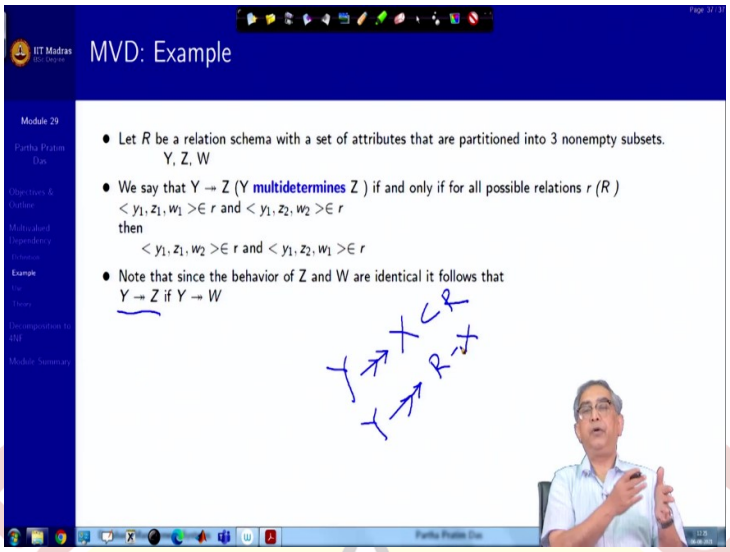
So, here is an example where I have two MVDs, course multi determines book and course multi determines lecturer. A course may have multiple so, saying that a course may have multiple books recommended for it and the course may be instructed by more than one lecturer. So, if you look into what is written here as t3 t4, what you'll find that basically what you have here you have two values each Silbertschatz, and Nederpelt.

You have two values here, John and William, but what you will get is you will get all combinations of the Cartesian product of them and that that precisely is this definition. So, if you take these as say, if if you take that the course multi determines this. So, this is t1 this is t2. So, they match on alphas. So, you will get two more t3 t4 who match on alpha, t3 beta should equate t1 beta.

What is t3 beta? t3, this book t3 beta should equate t1 beta t3[R-beta] will equate t3[R-beta] will equate t2[R-beta] like this. It is nothing but actually saying that all the Cartesian product items will come in here. So, that is the formal mathematical definition which can be used to reason on the MVDs. And naturally, as you have an MVD on course to book you have another MVD on course to lecture that will always be there.

(Refer Slide Time: 12:49)

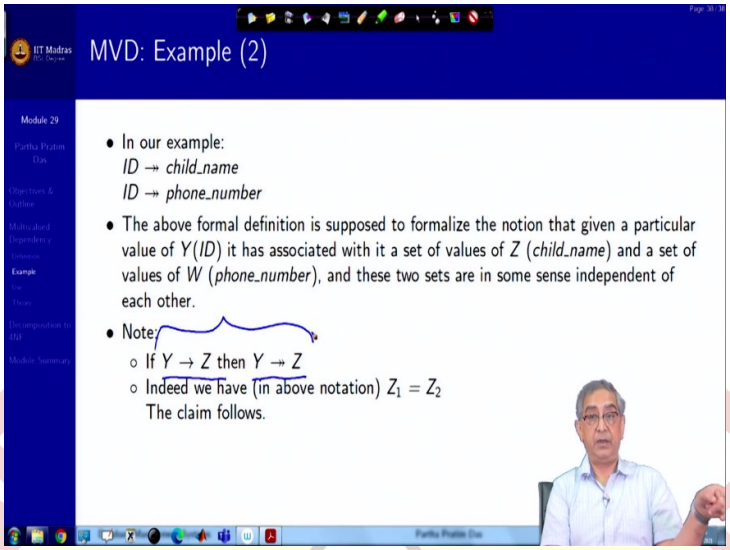


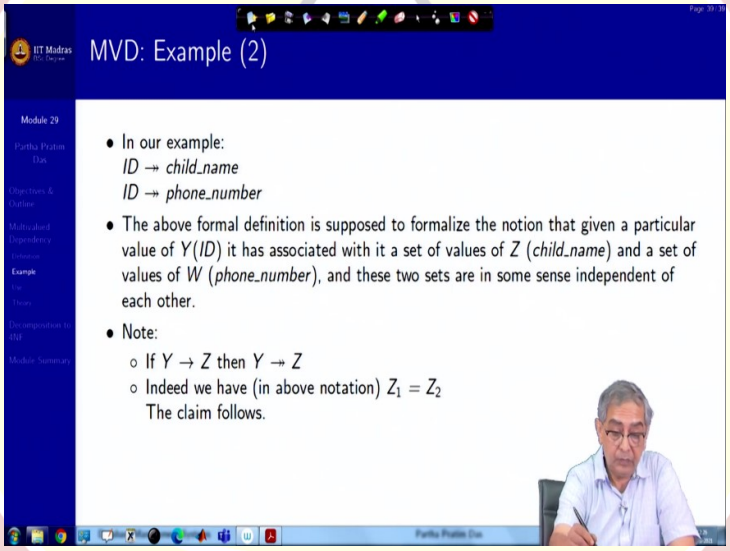


So, if we have a relational schema with a set of attributes, which is partitioned into three non-empty subsets Y, Z and W, there is partition. So, together it will from form R and they are disjointed. Then you say that, if Y multi determines Z if for all possible relations, if y1 z1 w1 and y1, z2, w2 both exist in R, then the Cartesian product combinations will also exist y1 z1, w2 and y2 y1, z2 w1 that is and we decide in a different way here, this is a partition of R.

So, since it is so, so, you can clearly see that, if you if you just have such a partition, then what it will mean that if you have a MVD from Y to Z, then you will also have an MVD to Y to W. So, what we say is if you have an MVD to Y to say X some subset of R, then you will also have an MVD to Y to R-X that is the partition. So, that is what you observe in this that is that is what you saw in the previous one also. We had course to book there are three attributes course, book and lecturer, a course to book MVD and therefore I have a course to lecture MVD or vice versa. So, this is a generalization of that notion.

(Refer Slide Time: 14:33)

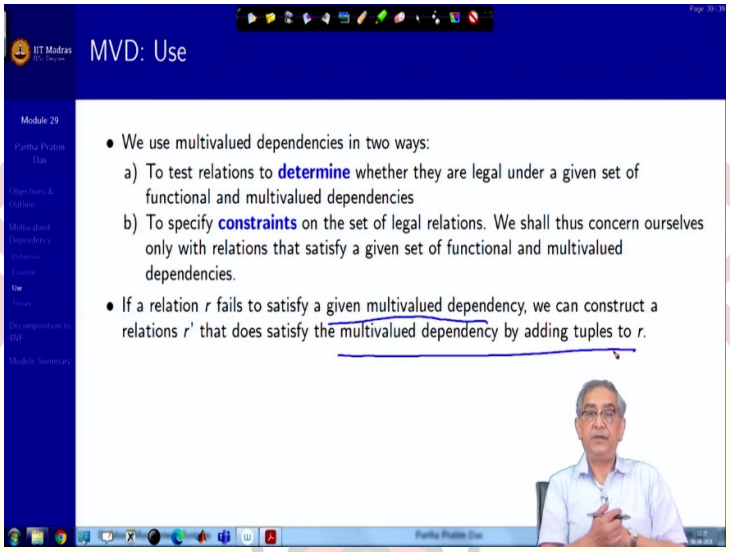




So again, similar statements for ID and child name. If you have ID to child name you will also have child name to ID to child name will also have ID to phone number. Now, if we say that Y functionally determines Z, Y functionally determines Z, then we will always have that Y multi determined Z, because all that is happening is that the two sets are actually identical, you can consider them to be identical.

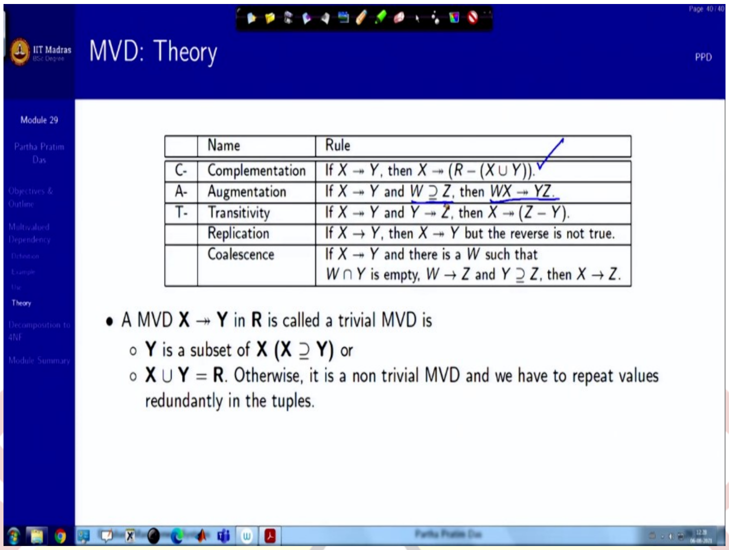
Here you are having Z you having W, they are basically identical, just two partitions. So, this is a golden rule to understand that if I have a functional dependency, I always have a corresponding multivalued dependency, but not the reverse the reverse is not true from the multivalued dependency I cannot come to functional dependence but this way it is true.

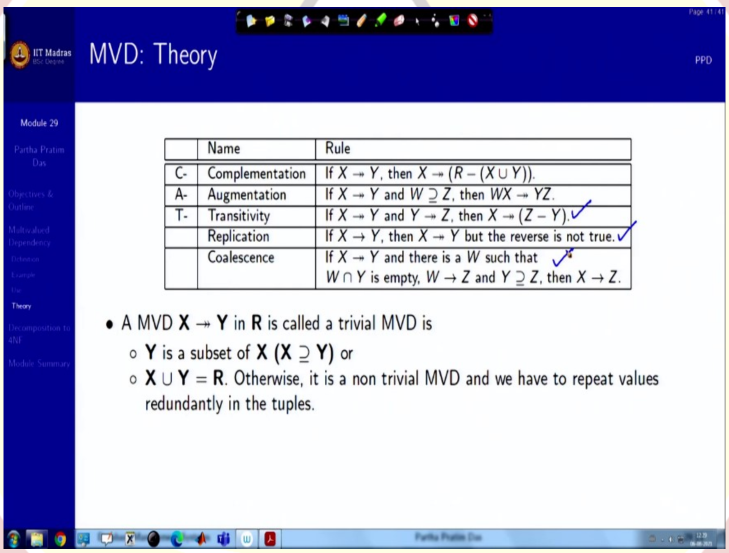
(Refer Slide Time: 15:48)

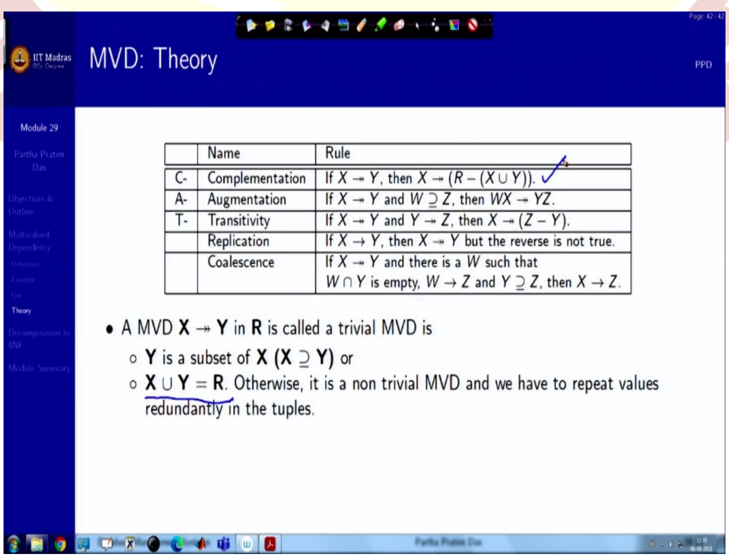


So, the how do you I mean what way do you use that possibly two ways one is test relations to determine whether they are legal under a set of functional and multivalued dependency and to specify constraints on the legal relation. Because if a relation fails to satisfy multivalued dependency, then we can always add the because those tuples are valid. But if an instance those are not there, then we can always add those tuples to satisfy the multivalued dependency.

(Refer Slide Time: 16:29)





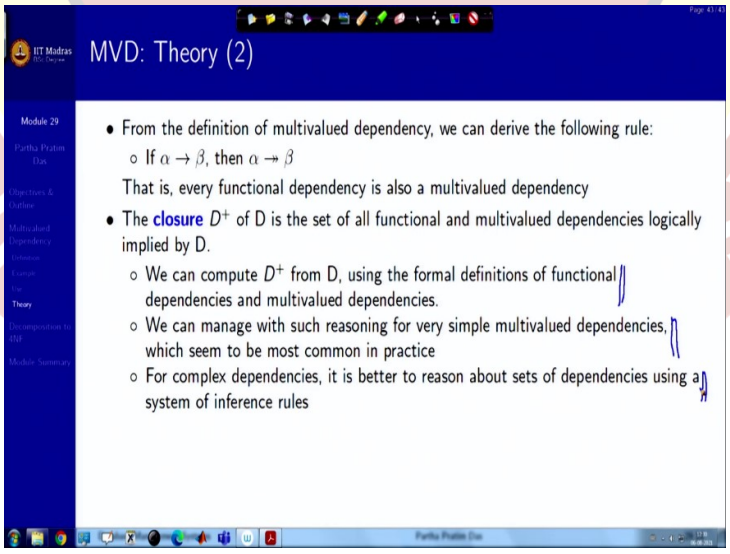


Now, likely we had Armstrong's actions, we have similar inferencing rules for MVDs as well. For example, of that, a special case of this is what we have seen, in general it is if R is X multi determines Y then X multi determines R- X union Y. So, you are creating that partition. Augmentation rule of functional dependencies gets stronger because now you do not need to augment with the same set on both sides, but left side maybe augmented with a larger set and the right side could be a smaller set as well.

The transitivity takes a different look as you can see, so, these you will have to, these are not difficult, you will have to sit down take examples or take the definition and get convinced that these rules are what hold. So, if X multi determines Y and Y multi determine Z then X multi determines Z-Y. Earlier it was Z alone. Now, it is Z-Y. Similarly, for replication this we have just said, in another coalescence.

So, these are different inferencing rules like the Armstrong's actions we had for multi valued dependencies. The notion of triviality also changes earlier we said that a functional dependency is trivial, trivial, if the left-hand, right-hand side is a subset of the left-hand side. But now that holds but in addition, it is also trivial if the union of the left-hand right-hand side is a whole set. Because of you already have this kind of rule. So, by that you will say that X union Y if it is R, then it is then X multi determines Y is a trivial multivalued dependency.

(Refer Slide Time: 18:43)

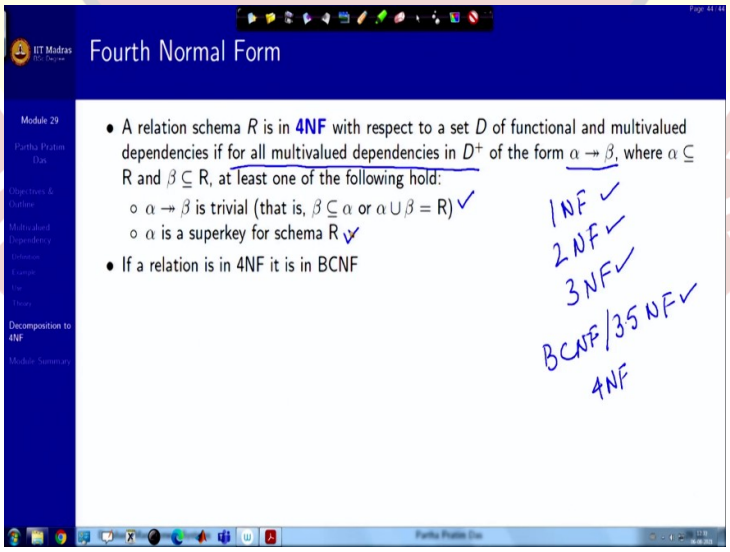


So, what we now get into actually using it in the normalization. So, we have the first following rule that if alpha determines beta than alpha multi determines beta. And in the same way that we did the closure of functional dependencies, we can now do the closure of all

functional as well as multivalued dependencies. We can compute them by using the formal definition.

In many cases, the situations of multiple values are not that complex. So, we can manage by doing simple reasoning, as we have been doing now. But if the situations are complex, then we will have to use inferencing rules that I just showed, and reason that to get to the closure of a set of functional as well as multivalued dependency and the concepts remaining concepts remain all the same. It is just that the dependency is being being relaxed in a different way.

(Refer Slide Time: 19:56)



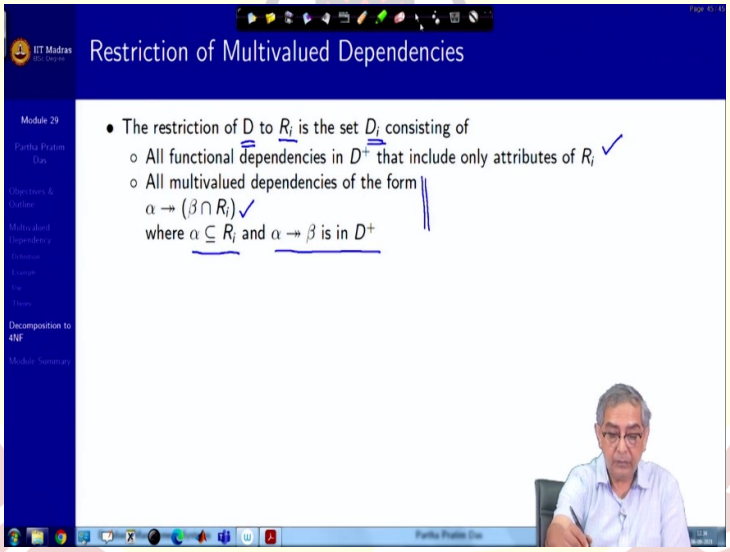
So, now, we will define and show the decomposition in the presence of multivalued dependency. So, like BCNF, 3NF and so on, we say that a relational schema is in 4NF. So, we have we have 1NF we have 2NF we have 3NF we have BCNF which many people call as

3.5NF and now, we have 4NF. Atomic value, partial dependency, transitive dependency and this is superkey restriction.

So, what do you how you define 4NF? Say that with respect to the set of functional and multivalued dependencies if for all multivalued dependencies in D plus that is a closure set of the form alpha multi determines beta at least one of the two will have to hold. One is either it is a trivial MVD or is a superkey, superkey of the schema.

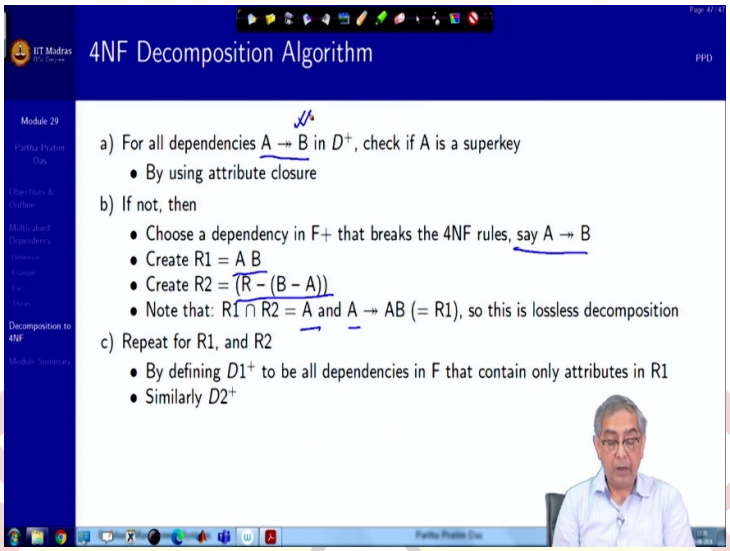
So, only thing that you are injecting is a notion of multivalued dependency but at the end, the definition of 4NF is pretty much like BCNF and it is it will be quite obvious to see that if a relation is in 4NF, it is necessarily in BCNF because every functional dependency is a multivalued dependency as well. So, again the conditions are, every multivalued dependency is either trivial or the left-hand side is the superkey of the schema.

(Refer Slide Time: 22:08)



Now, to be able to check for different things, you will need to define the restriction in the same way. So, if you have decomposed a relation to Ri, then you have to restrict D the set of functional and multivalued dependencies to Di which has all the functional dependencies in D plus that include only attributes of Ri this is what we have been doing earlier. And this is the addition that it will have all multivalued dependencies of the form beta in alpha multi determines beta intersection Ri, where alpha is in Ri, and this is in D plus. That is quite clear.

(Refer Slide Time: 23:17)

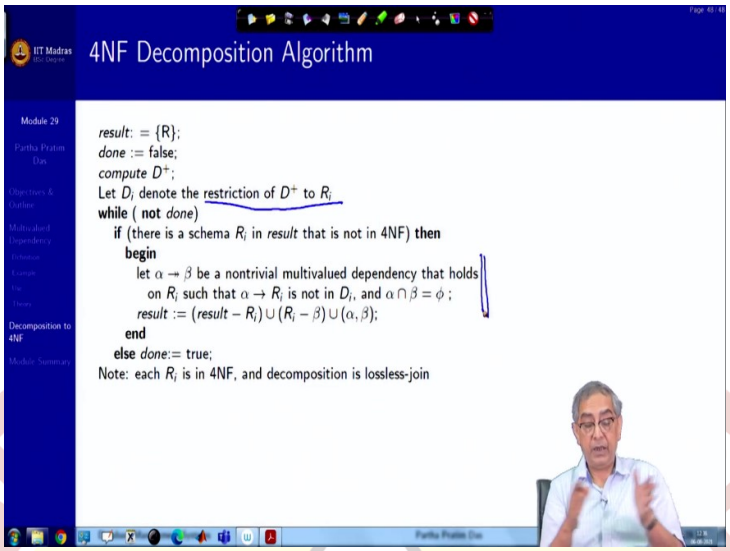


Now, if a relationship is not in 4NF, we will again do the same we will decompose to make it into 4NF. And it goes exactly like what we did for BCNF. So, we will check using attribute closure defined in the same way as we did for functional dependency. Whether A multi determines B is in the closure set if it is not, if this from this closure set, we will know whether it is a, A is a super key or not.

If A is not a super key, then we will have to create a decomposition and in the same way, we will have A union B in one decomposition and A minus B minus in other decomposition. Mind you the consequent observation remains same that the intersection is A and therefore you will have a lossless join. There is now no problem with that. So, this is exactly like the BCNF except that you are doing the closure in terms of the multivalued dependencies.

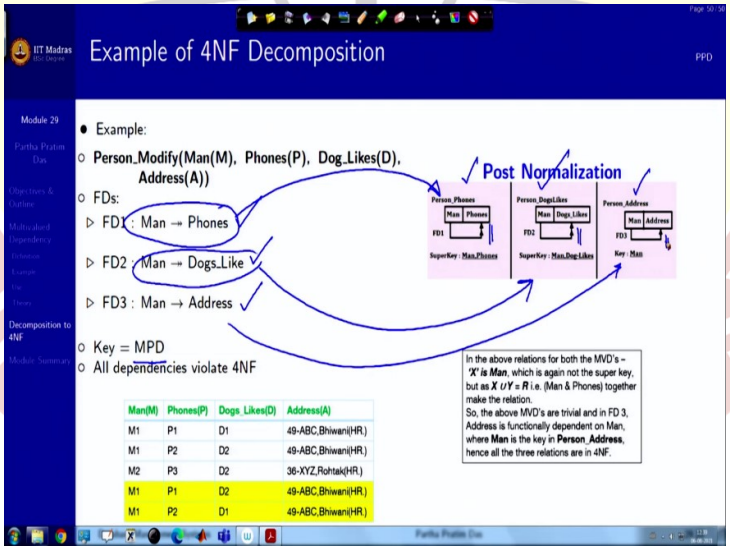
Now, after you have got R1 and R2, we have to check each if they are in 4NF. And for checking that we have to create those restrictions D1 and D2 and check within D1, the closure of D1 and closure of D2 and if any one or both of them are not in multivalued or not in 4NF, then I will have to again go forward and decompose it further.

(Refer Slide Time: 25:13)



So, formally said this is the pseudocode of the algorithm, which you can go through and understand in steps, but what you have said is the basic process. So, you have the restrictions being computed that is the first thing, restrictions being computed, and then you are doing the checking. And based on that you do the partitioning of the relational schema.

(Refer Slide Time: 25:43)



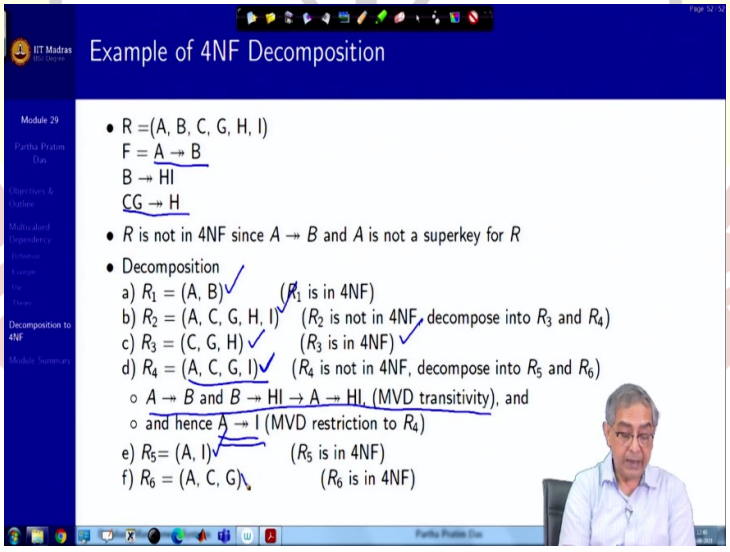
So, now we have learned how to create 4NF so, let us look at an example. So, let us go back to our person relational schema, all that I have done is I have included another attribute, which is not multivalued, it is a single valued, a person has one address, multiple phones, multiple dog likes. So, there is a I mean, it is written as FD this basically dependencies.

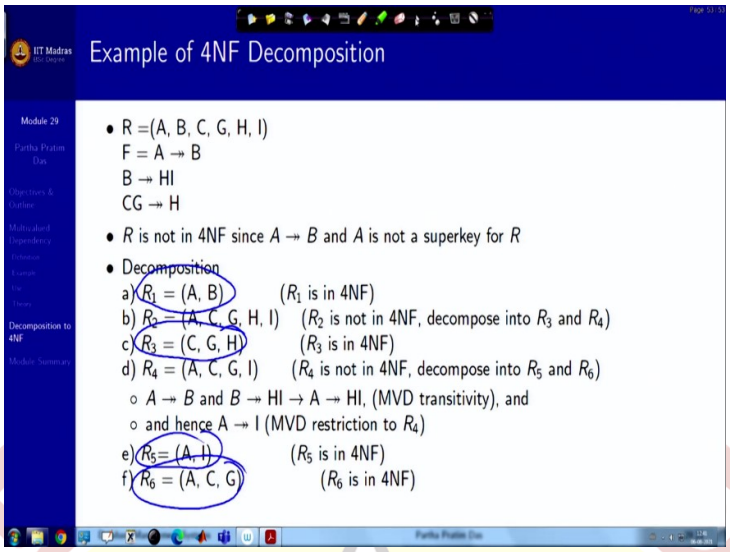
So, first one is an MVD, man multi determines phones, can have multiple phones, can have multiple likes to dogs man multi determines dog and has a unique address naturally given this all the three attributes are forming the key. So, here as you can see that all dependencies violate 4NF because none of them are key on the left-hand side and none of them are trivial. So, what we would do?

We would use this violation and create a decomposition here which is person phone which is man and phone, the remaining will also so this is what will remain is person, dog like, address that will also violate. So, we will take this, create this and we will be left with the last one. So, here in this relation, we have one multivalued dependency in the restriction here we have another multivalued dependency in the restriction, here we have the functional dependency and the restriction you can see the notations here.

FD1, FD2, FD3. So, with this normalization, the entire data schema comes into fourth normal form. So, it has minimal amount of redundancy, it naturally through the process has lossless joint and as it so happens that all the 3, FD or MVD are shakeable in the respective decomposed relation. Therefore, this is also dependency preserving decomposition into 4NF incidentally.

(Refer Slide Time: 28:29)

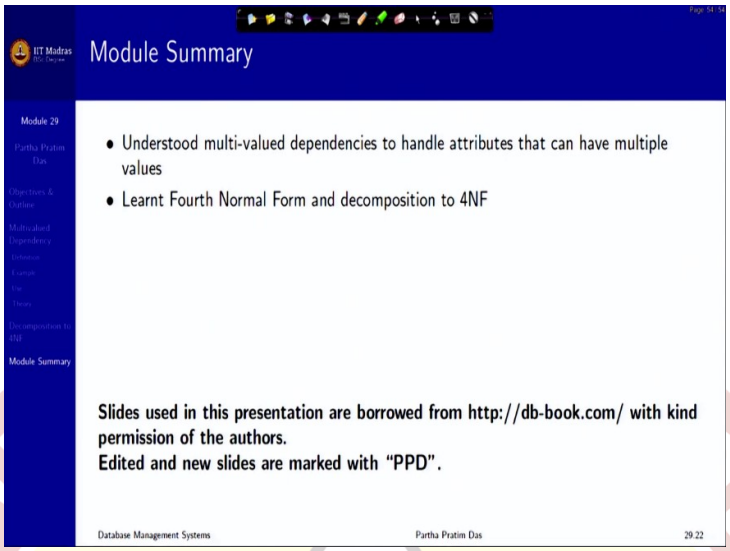




This is a longer example where we have started with considering this A, B, I mean A multi determines B which is which violates, so I have A B and I have the remaining A, C, G, H, I. Now, this is in 4NF clearly but A, C, G, H, I is not in 4NF because it will have C, G multi determines H. As we have that so we again partition C, G, H and this C, G, H is in 4NF but A, C, G, I is not.

To see that A, C, G, I is not we have to actually get into the closure of the set of functional multivalued dependencies. And you can see using the MVD transitivity that there is a multivalued dependency A multi determining I which is available in the restriction R4. So, naturally that will get violated. So, we have to partition R4 again, I get A, I, I get A, C, G. So finally, R1, R3, R5, R6 these four together give me a 4NF decomposition of the original relation.

(Refer Slide Time: 30:11)



So, in this module, as we conclude, we have understood the additional redundancy factors in the design of relational schema and multi valued attributes situation we have tried to address and seen that I can all we can take a schema into 4NF through decomposition, ensuring lossless joint, it may or may not preserve dependencies. In reality, I would say that it is much less frequent that you want users the 4NF because the prevalence of multiple valued attributes are not that huge.

Or maybe you will take care of those but they are very typical like phone number like address and so on. So, we will take care of those in that way. But this is the general founding theory for this and it is also that this does not mean that 4NF gets rid of all kinds of redundancies. There are other types of redundancies like joint redundancy and so on for each 5NF and so on so forth.

There are several other normal forms higher and higher, which removes more and more redundancies. But we will stop here on the theory of design of relational databases, because those are not very frequently used. Thank you very much for your attention and we meet in the next module.